Angular Components

Lecture 5

Component Architecture • Data Binding • Lifecycle Hooks

KBTU — Web Development

1. Angular Components — Deep Dive

In Lecture 4, we introduced Angular and created our first component. Now we go deeper. Components are the fundamental building blocks of every Angular application. Each component controls a piece of the screen called a **view**. A typical Angular app is a tree of components, starting from a root component.

1.1 What Is a Component?

A component is a TypeScript class decorated with `@Component()`. It combines three things:

- **Template** — the HTML that defines what renders on screen
- **Class** — the TypeScript code that handles data and logic
- **Styles** — the CSS that controls appearance (scoped by default)

1.2 Component Anatomy

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-student-card',       // HTML tag name
  templateUrl: './student-card.html',  // external template
  styleUrl: './student-card.css'       // external styles
})
export class StudentCardComponent {
  // Properties (data)
  name: string = 'Alice';
  grade: number = 95;

  // Methods (behavior)
  isPassing(): boolean {
    return this.grade >= 60;
  }
}
```

1.3 Component Metadata (@Component Decorator)

Property	Description	Example
selector	CSS selector that identifies this component in templates	'app-student-card'
templateUrl	Path to external HTML template file	'./student-card.html'
template	Inline HTML template (for short templates)	`<h1>Hello</h1>`
styleUrl	Path to external CSS file	'./student-card.css'
styles	Inline CSS styles array	[`h1 { color: red }`]
imports	Standalone components/modules used in this component	[FormsModule, CommonModule]
standalone	Whether standalone (default: true in v17+)	true

1.4 Creating Components with Angular CLI

```
# Generate a new component
ng generate component student-card
# Shorthand
ng g c student-card

# This creates:
# src/app/student-card/
#   student-card.ts         (component class)
```

```
#   student-card.html       (template)
#   student-card.css        (styles)
#   student-card.spec.ts    (unit test)
```

The CLI automatically creates all four files. In Angular 17+, components are **standalone by default** — no NgModule declaration needed.

2. Component Properties and Data Binding

Angular provides four forms of data binding that control how data flows between the component class and the DOM.

2.1 The Four Types of Data Binding

Type	Direction	Syntax	Use Case
Interpolation	Component → View	{{ expression }}	Display text content
Property Binding	Component → View	[property]="expr"	Set element properties
Event Binding	View → Component	(event)="handler()"	Respond to user actions
Two-way Binding	Both directions	[(ngModel)]="prop"	Form inputs synced to data

2.2 Interpolation {{ }}

Interpolation evaluates an expression and converts the result to a string. It works inside text content.

```
<!-- Simple property -->
<h1>Hello, {{ studentName }}!</h1>

<!-- Expressions -->
<p>Total: {{ price * quantity }}</p>
<p>Full name: {{ firstName + ' ' + lastName }}</p>

<!-- Method calls -->
<p>Status: {{ isPassing() ? 'Pass' : 'Fail' }}</p>

<!-- Cannot use: assignments, new, chaining, ++ / -- -->
```

2.3 Property Binding [ ]

Property binding sets an element property to the value of a component expression.

```
<!-- Set image source dynamically -->
<img [src]="student.photoUrl" [alt]="student.name">

<!-- Disable button based on condition -->
<button [disabled]="!isFormValid">Submit</button>

<!-- Set CSS class conditionally -->
<div [class.active]="isSelected">Card</div>

<!-- Set inline style -->
<div [style.background-color]="bgColor">Styled</div>
```

2.4 Event Binding ( )

Event binding listens for DOM events and calls a component method when triggered.

```
<!-- Click event -->
<button (click)="onSave()">Save</button>

<!-- Pass the event object -->
<input (input)="onInput($event)">

<!-- Keyboard events -->
<input (keyup.enter)="onSubmit()">
```

2.5 Two-way Binding [( )]

Two-way binding combines property binding and event binding. Requires `FormsModule` to be imported.

```
import { FormsModule } from '@angular/forms';

@Component({
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="searchTerm" placeholder="Search...">
    <p>Searching for: {{ searchTerm }}</p>
  `
})
export class SearchComponent {
  searchTerm: string = '';
}
```

3. Parent–Child Component Communication

In a real application, components need to talk to each other. Angular provides **`input()`** to pass data from parent to child, and **`output()`** to send events from child to parent.

3.1 Passing Data Down with input()

New Signal-Based Syntax (Preferred)

```
// child: student-card.ts
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-student-card',
  template: `
    <div class="card">
      <h3>{{ name() }}</h3>
      <p>Grade: {{ grade() }}</p>
    </div>
  `
})
export class StudentCardComponent {
  name = input.required<string>();   // required input
  grade = input<number>(0);          // optional with default
}
```

Legacy Decorator Syntax (Still Works)

```
import { Component, Input } from '@angular/core';

@Component({ ... })
export class StudentCardComponent {
  @Input() name!: string;      // required (use !)
  @Input() grade: number = 0;  // optional with default
}
```

Parent Usage

```
<!-- parent template -->
<app-student-card [name]="student.name" [grade]="student.grade" />

<!-- Using @for to render multiple cards -->
@for (student of students; track student.id) {
  <app-student-card [name]="student.name" [grade]="student.grade" />
}
```

3.2 Emitting Events Up with output()

New Syntax (Preferred)

```
import { Component, input, output } from '@angular/core';

@Component({
  selector: 'app-student-card',
  template: `
    <div class="card">
      <h3>{{ name() }}</h3>
      <button (click)="onDelete()">Delete</button>
      <button (click)="onLike()">Like ({{ likes }})</button>
    </div>
  `
})
export class StudentCardComponent {
  name = input.required<string>();
  delete = output<number>();    // emits student id
  liked = output<void>();       // emits no data

  likes = 0;
```

```
  onDelete() { this.delete.emit(this.id); }
  onLike() { this.likes++; this.liked.emit(); }
}
```

Parent Listens to Child Events

```
@for (student of students; track student.id) {
  <app-student-card
    [name]="student.name"
    (delete)="removeStudent($event)"
    (liked)="onStudentLiked()" />
}

// parent class
removeStudent(id: number) {
  this.students = this.students.filter(s => s.id !== id);
}
```

3.3 Data Flow Summary

Direction	Mechanism	New Syntax	Legacy Syntax
Parent → Child	Input	input() / input.required()	@Input() decorator
Child → Parent	Output	output()	@Output() + EventEmitter

Key Rule: Data flows DOWN via inputs, Events flow UP via outputs. This is called **unidirectional data flow** and is a core Angular architecture principle.

4. Templates and Styles

4.1 Template Expressions

Template expressions produce a value for a binding target. Keep them simple, side-effect-free, and fast.

- **Allowed:** property access, method calls, basic operators (+, -, *, /, %, ternary)
- **NOT allowed:** assignments (=), new, chaining (;), ++ / -- operators
- Use the **safe navigation operator** (?.) to guard against null/undefined

```
<!-- Safe navigation operator -->
<p>{{ student?.address?.city }}</p>

<!-- Nullish coalescing -->
<p>{{ student?.name ?? 'Unknown' }}</p>
```

4.2 Template Reference Variables

A template reference variable (#var) gives you a reference to a DOM element within the template.

```
<input #nameInput type="text" placeholder="Enter name">
<button (click)="greet(nameInput.value)">Greet</button>

// In the component:
greet(name: string) { alert('Hello, ' + name); }
```

4.3 Component Styles & View Encapsulation

Angular scopes component styles by default. Styles defined in a component only affect that component's template.

Mode	Behavior	Use Case
Emulated (default)	Angular emulates Shadow DOM by adding unique attributes	Most components
ShadowDom	Uses native Shadow DOM (full style isolation)	Web components
None	No encapsulation — styles are global	Global utilities (use sparingly)

Special CSS Selectors

```
/* :host — targets the component's own element */
:host {
  display: block;
  padding: 16px;
  border: 1px solid #ddd;
}

/* :host(.active) — when the host has a class */
:host(.highlighted) { border-color: #dd0031; }

/* ::ng-deep — pierces encapsulation (use sparingly!) */
:host ::ng-deep .child-class { color: red; }
```

5. Component Lifecycle Hooks

Every Angular component has a lifecycle managed by Angular. Lifecycle hooks are special methods that let you tap into key moments.

Hook	Called When	Common Use
constructor()	Class instantiated (before Angular)	Dependency injection only
ngOnChanges()	Input property value changes	React to input changes
ngOnInit()	After first ngOnChanges	Fetch data, initialize logic
ngDoCheck()	Every change detection run	Custom change detection (rare)
ngAfterContentInit()	After projected content init	Access projected content
ngAfterContentChecked()	After projected content checked	React to content changes
ngAfterViewInit()	After view initialized	Access child views / DOM
ngAfterViewChecked()	After view checked	React to view changes
ngOnDestroy()	Before component is destroyed	Cleanup: unsubscribe, timers

5.1 ngOnInit — Component Initialization

Called once, after the first ngOnChanges(). This is where you put initialization logic.

```
import { Component, OnInit } from '@angular/core';

@Component({ ... })
export class StudentListComponent implements OnInit {
  students: Student[] = [];

  ngOnInit(): void {
    console.log('Component initialized!');
    this.students = this.loadStudents();
  }

  private loadStudents(): Student[] {
    return [
      { id: 1, name: 'Alice', grade: 95 },
      { id: 2, name: 'Bob', grade: 82 },
    ];
  }
}
```

5.2 ngOnChanges — React to Input Changes

Called whenever an input property changes. Receives a SimpleChanges object.

```
import { Component, input, OnChanges, SimpleChanges } from '@angular/core';

@Component({ ... })
export class StudentCardComponent implements OnChanges {
  grade = input.required<number>();
  gradeStatus: string = '';

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['grade']) {
      const val = changes['grade'].currentValue;
      this.gradeStatus = val >= 60 ? 'Passing' : 'Failing';
    }
  }
}
```

5.3 ngOnDestroy — Cleanup

Called once, right before Angular destroys the component. Release resources here.

```
import { Component, OnDestroy } from '@angular/core';

@Component({ ... })
export class TimerComponent implements OnDestroy {
  private intervalId: any;

  constructor() {
    this.intervalId = setInterval(() => console.log('tick'), 1000);
  }

  ngOnDestroy(): void {
    console.log('Cleaning up!');
    clearInterval(this.intervalId);
  }
}
```

6. Practical Demos

Demo 1: Multi-Component Task App

Building on Lecture 4's task app, we refactor it into parent-child components. The parent manages the task list. A child component displays each task and emits delete events.

task-item.ts (Child Component)

```
import { Component, input, output } from '@angular/core';

@Component({
  selector: 'app-task-item',
  templateUrl: './task-item.html',
  styleUrl: './task-item.css'
})
export class TaskItemComponent {
  task = input.required<string>();
  index = input.required<number>();
  delete = output<number>();

  onDelete() { this.delete.emit(this.index()); }
}
```

task-item.html (Child Template)

```
<li class="task-item">
  <span>{{ task() }}</span>
  <button (click)="onDelete()" class="delete-btn">Delete</button>
</li>
```

Updated app.html (Parent Template)

```
<h3>Task APP (Multi-Component)</h3>
<input type="text" [(ngModel)]="newTask">
<button (click)="addTask()">Add Task</button>

<ul>
  @for (task of tasks; track $index) {
    <app-task-item
      [task]="task"
      [index]="$index"
      (delete)="deleteTask($event)" />
  } @empty {
    <li>No tasks yet!</li>
  }
</ul>
```

Demo 2: Lifecycle Hooks in Action

```
import { Component, OnInit, OnDestroy, input, OnChanges,
         SimpleChanges } from '@angular/core';

@Component({
  selector: 'app-student-card',
  template: `
    <div class="card" [class.highlight]="isNew">
      <h4>{{ name() }} — Grade: {{ grade() }}</h4>
      <p>Status: {{ status }}</p>
      <p>Time on screen: {{ timer }}s</p>
    </div>
  `
})
export class StudentCardComponent
  implements OnInit, OnChanges, OnDestroy {

  name = input.required<string>();
  grade = input<number>(0);
```

```
  status = ''; isNew = true; timer = 0;
  private intervalId: any;

  ngOnInit(): void {
    this.intervalId = setInterval(() => this.timer++, 1000);
    setTimeout(() => this.isNew = false, 2000);
  }

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['grade']) {
      const g = changes['grade'].currentValue;
      this.status = g >= 90 ? 'Excellent' :
                    g >= 70 ? 'Good' : 'Needs Help';
    }
  }

  ngOnDestroy(): void {
    clearInterval(this.intervalId);
  }
}
```

7. Practice Exercises

Exercise 1: Create a ProductCardComponent that receives a product object via input() and displays its name, price, and image. Use it in AppComponent with @for to render 5 products.

Exercise 2: Add a 'Delete' button to ProductCardComponent. Use output() to emit the product ID. The parent should remove the product from the list when the event fires.

Exercise 3: Add a 'Like' button to each product card. Track likes locally in the child. Emit a liked event to the parent that logs which product was liked.

Exercise 4: Implement ngOnInit in ProductCardComponent to log when each card is created. Implement ngOnDestroy to log when it is removed. Delete products and observe the console.

Exercise 5: Create a component that starts a timer in ngOnInit (using setInterval). Display the elapsed seconds. Use ngOnDestroy to clear the interval. Toggle the component with @if.

Exercise 6: Create a category selector. When a category is clicked, display its products using a child component. Use ngOnChanges in the child to detect when the list changes.

Summary

Topic	Key Takeaways
Components	TypeScript class + template + styles; created via ng g c
Data Binding	Interpolation {{ }}, property [ ], event ( ), two-way [( )]
Parent → Child	input() / input.required() passes data down
Child → Parent	output() emits events up to the parent
Styles	Scoped by default (ViewEncapsulation.Emulated); :host selector
Lifecycle	ngOnInit (setup), ngOnChanges (react), ngOnDestroy (cleanup)
Best Practice	Keep components small, focused; data down, events up

References

- Angular Components Guide
- Component Inputs
- Component Outputs
- Lifecycle Hooks
- View Encapsulation